CPD: Project 1

March 2024

Grupo T11G13

Pedro Madureira - up202108866
Sofia Pinto - up202108682
Tomás Gaspar - up202108828

Problem description and algorithms explanation

Problem 1 - Matrix Multiplication

The first problem of this assignment was already partially implemented for us. The goal was to multiply two matrices, the traditional way. As the C++ implementation had already been provided, we focused on rewriting the code, but this time using a higher level programming language: Java.

The code we developed turned out to be really similar to the original one. It basically consisted in three chained '*for*' loops which will then apply the multiplication of each row with the corresponding column.

Problem 2 - Line Matrix Multiplication

The second problem's purpose was to develop a more efficient approach to the previous one, by avoiding several cache misses.

The previous algorithm used the conventional matrix multiplication technique, which does not take advantage of the temporal and spatial locality within the cache. This limitation happens due to the extensive memory address jumps involved, which are typically larger than the blocks of memory the cache keeps, thereby increasing cache misses.

To address this issue, we modified the implementation to traverse both matrixes sequentially, that is, row by row. This allowed us to incrementally calculate the result of each position in the resultant multiplied matrix.

Additionally, we are required to initialise the resultant matrix with zeros as a step in the initialization process.

Problem 3 - Block Matrix Multiplication

Since block matrix multiplication involves splitting the original matrices into blocks and performing multiplications on those blocks as if they were individual elements, our solution to this problem is similar to the previous one.

Initially, we focused on the high-level by multiplying blocks. We now make jumps with the same size as the blocks (bkSize) given we are treating each block as a separate element:

```
for (i = 0; i < mSize; i+=bkSize) {
     for (j = 0; j < mSize; j+=bkSize) {
          for (k = 0; k < mSize; k+=bkSize) {
               c[i,j] += MatrixMult(a[i, k], b[k, j]);
               }
          }
     }
```

Code 1: Block Matrix Multiplication pseudocode

Then, we also adapted the code from the second solution to create the MatrixMult function depicted above.

Performance metrics

All the tests were conducted on a laptop with a 16-thread AMD CPU, connected to a power supply, and running the Arch Linux operating system. The C++ versions of the algorithms were compiled with g++, and the flag '-O2', and PAPI (Performance Application Programming Interface) was used to measure L1 and L2 data cache misses. For non-parallel algorithms, the time was measured using libc's clock function, while for parallel versions, omp_get_wtime from OpenMP was used.

In addition to cache misses, another key metric analysed was FLOPS (FLoating-point Operations Per Second) and it was calculated using $2 \times n^3 \div time$, in which *n* is the size of the matrix under consideration and *time* denotes the duration taken to complete the calculation. This formula can be derived from the understanding that there are 3 nested '*for*' loops each iterating through n elements, and each iteration of the inner loop performs a floating point addition and multiplication.

Results and analysis

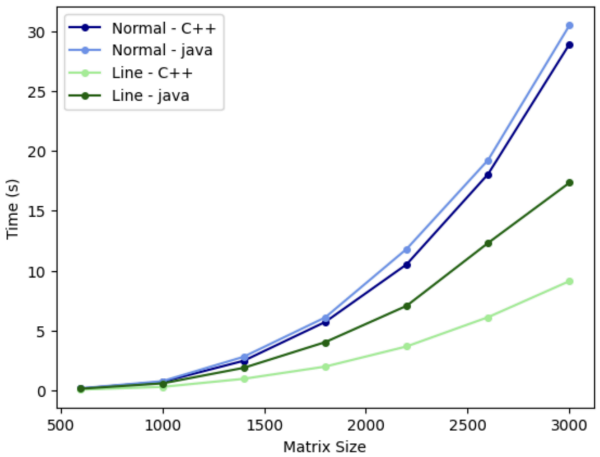
Below, we present the obtained results by applying our different solutions to the previous problems.

Problem 1 and 2 - Matrix Multiplication and Line Matrix Multiplication

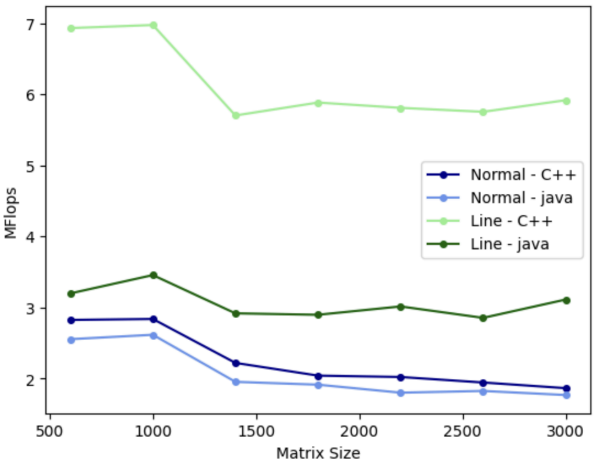
For the first two implementations of the matrix multiplication algorithms, the time complexity of both problems is O(n^3), and the space complexity is O(n^2), where *n* is the dimension of the matrix. Yet, the performance depends on the algorithm used, even for equal sized matrices, as we will see in this section. Here, we will compare the execution time of the first two algorithms and its corresponding implementation in different programming languages.

Given C++ is compiled, there is a translation of the code into machine code specific to the processor before running. In contrast, Java is an interpreted language, where the code is translated line by line during runtime by the Java Virtual Machine (JVM), which is more time-consuming than C++. In addition, it uses automatic garbage collection, which frees memory no longer in use, but may affect performance, as the memory management is not fully controlled by the user. [1]

Therefore, it is reasonable to state that the identical implementation will typically be faster in C++ compared to Java. [2] The graphs below corroborate this affirmation, showing that multiplication algorithms in C++ outperform their implementations in Java, particularly with larger matrix sizes.



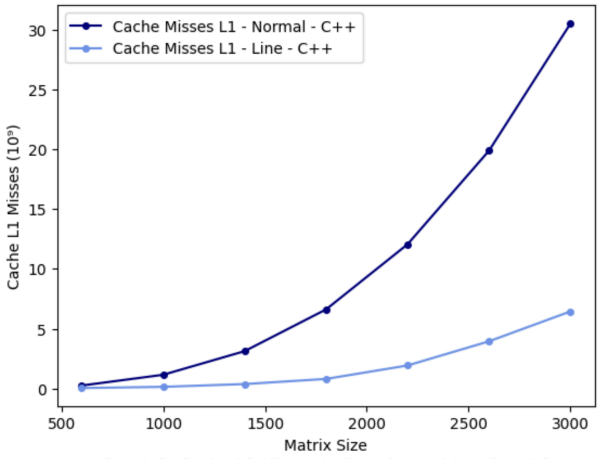
Graph 1: Execution time for problem 1 and 2



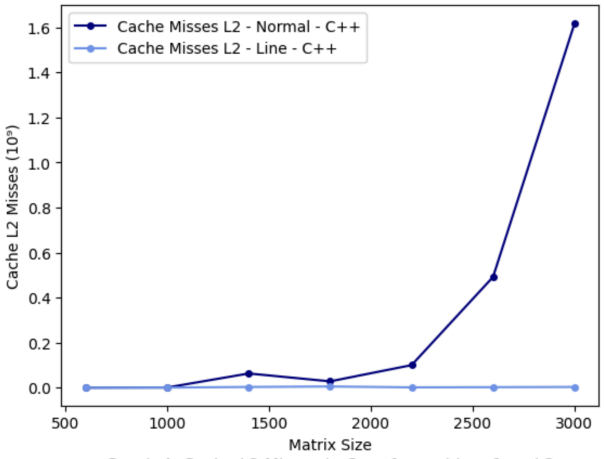
Graph 2: FLOPS for problem 1 and 2

Now focusing our attention solely on the C++ implementations of the problems, we can observe two main aspects: the amount of cache misses is significantly lower in Cache L2, as well as in the Line Matrix Multiplication Algorithm. These conclusions are comprehensible, due to the fact that the L1 Cache is much smaller in size when compared to L2. The lower

capacity means that it can hold less data and instructions, which increases the likelihood of cache misses, as depicted below. Moreover, the Line Matrix Multiplication Algorithm produces much less cache misses than the previous implementation, due to our efforts to access the memory sequentially, which decreases the probability of a cache miss.



Graph 3: Cache L1 Misses in C++ for problem 1 and 2



Graph 4: Cache L2 Misses in C++ for problem 1 and 2

Problem 2 - Multi-core Implementation

The second part of this project encouraged us to implement two distinct parallel versions of our second solution (line x line) for the matrix multiplication problem, while achieving the same correct multiplication results.

The difference in the two proposed implementations lies in the fact that, in the first one, the outer loop is parallelized meaning that each thread executes a subset of the iterations of this loop independently, while in the second version, the inner loop is parallelized, i.e. each thread executes a subset of the iterations of the inner loop. In practice, what this means is that the overhead associated with parallelization occurs only once for the first version and every iteration of the two outer loops for the second version. Therefore, it is to be expected that the second version of the multi-core implementation is slower than the first version.
Below, we present two tables which depict the performance of these parallel implementations in terms of MFlops, speedup and efficiency:

	4096	6144	8192	10240
MFLOPS	23.574	23.148	22.162	24.652
Speedup	5.819	5.716	5.472	6.013
Efficiency (%)	36.4	35.7	34.2	37.6

Table 1: First parallel implementation performance for different matrix sizes

	4096	6144	8192	10240
MFLOPS	7.739	8.894	9.195	9.961
Speedup	1.91	2.196	2.27	2.429
Efficiency (%)	11.9	13.7	14.2	15.2

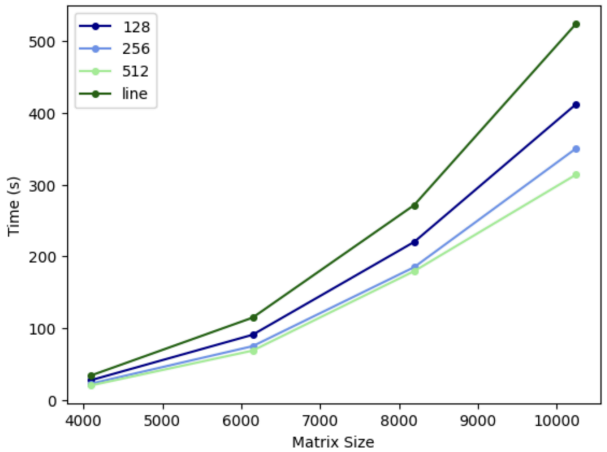
Table 2: Second parallel implementation performance for different matrix sizes

From the thorough examination of these tables, we can conclude that, as we had expected, the first parallel version is much better in terms of every performance metric we studied. However, both of these implementations evidence a Speedup value greater than 1, regardless of the matrix size. This means that both of them are still faster than the sequential version, despite the clear efficiency improvement that the first implementation provides.

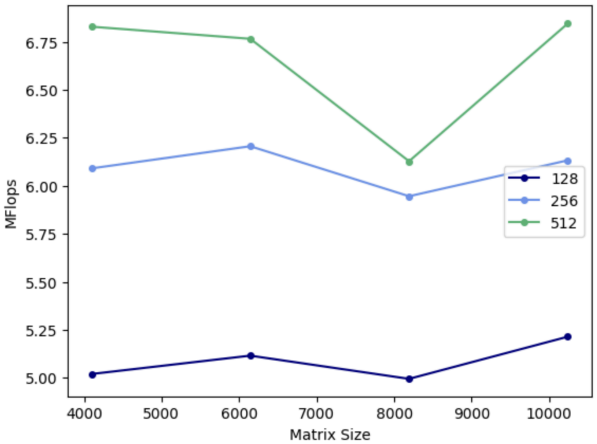
Problem 3 - Block Matrix Multiplication

For the last algorithm, we need to specify the value of the block size that we will use to divide the original matrix into smaller matrices. The choice of this value is crucial and will have an impact on the performance, as it will directly impact the cache misses. This will be better if the same block copied into cache can be used several times. [3]

Below, there are two graphs showcasing the variance of the execution time (Graph 5) and the FLOPS (Graph 6) for different matrix and block sizes. From the execution time graph, we can see similar results, especially for matrices of size 6144 and below, but overall the performance is better on blocks of 512, than 256 or 128. It is also significant to notice that this solution is faster than the previous one, regardless of the chosen block size. With this approach, we are breaking down the matrices into smaller blocks, which increases the likelihood of reusing data that is already in cache, reducing the number of cache misses, hence improving the overall memory access efficiency.



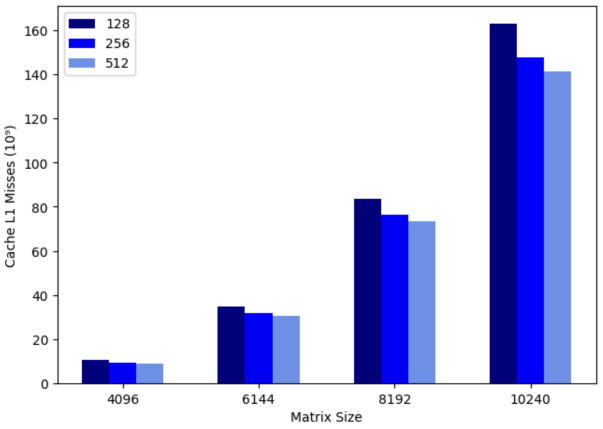
Graph 5: Execution time for problem 2 and 3



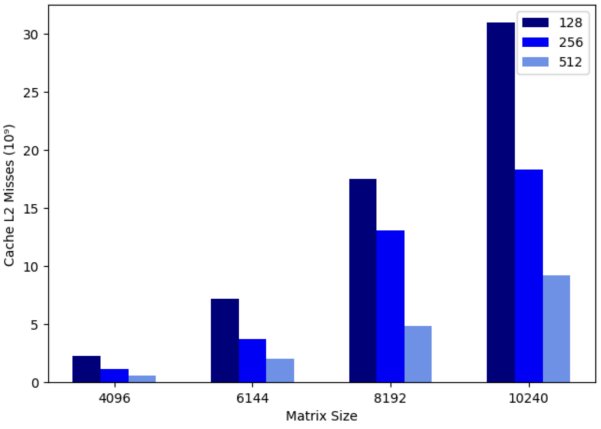
Graph 6: FLOPS for problem 3

In order to better see the differences, we can analyse the FLOPS for each block, from which we conclude the same results. Nevertheless, there is a small decrease for the size 8192, which can be attributed to limitations related to the system's architecture.

To further compare the differences between the blocks, we can also analyse the cache misses in the following graphs:



Graph 7: Cache L1 Misses for problem 3



Graph 8: Cache L2 Misses for problem 3

From the graphs, we can see that both on caches L1 and L2 the misses decrease with the size of the block. It is also worth noting that our program is robust, since it works smoothly even when the size of the matrix is not divisible by the size of the block.

In conclusion, we can depict from the graphs analysis that the overall performance is better when using this algorithm, particularly for matrices using blocks of size 512.

Conclusions

This project has given us the opportunity to implement multiple solutions for a well-known problem: Matrix Multiplication. We were able to witness the benefits of applying efficient algorithms, namely the block multiplication technique. Moreover, we understood that a multi-core approach brings even better results, especially when well implemented.

Finally, this project provided us with a practical and enriching learning experience enabling us to gain a deeper understanding of the underlying challenges involved when optimising and parallelizing low-level code.

References

[1] GeeksforGeeks, "C++ vs Java," GeeksforGeeks

URL: https://www.geeksforgeeks.org/cpp-vs-java/ (accessed March 13, 2024).

[2] Gherardi, Luca, Davide Brugali, and Daniele Comotti. 'A Java vs. C++ Performance Evaluation: A 3D Modeling Benchmark'. In Simulation, Modeling, and Programming for Autonomous Robots, 161–72. Springer Berlin Heidelberg, 2012.

URL: https://doi.org/10.1007/978-3-642-34327-8_17.

[3] Ristov, S., Gusev, M., & Velkoski, G. "Optimal block size for matrix multiplication using blocking." In 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), pp. 295-300. Opatija, Croatia, 2014.

URL: https://ieeexplore.ieee.org/document/6859580